

Section 1: About You

Goals and Objectives

Why do you want to do a GSoC project with Oppia?	Hi, I am Neer Aryan Bhatta and I have been building Android apps for almost three years now solving the problems that I run into, freelance work across different stack and side projects on my Github. But Oppia's Android codebase is unlike what I have worked with before, Bazel, protocol buffers, multi-module architecture with strict separation between domain, data, and UI layers, it was a steep learning curve but honestly this was what I enjoyed the most. Working on this repository I have learned more about production-grade Android engineering in a few months of contributing here than I did in a year of working on my own. A big part why anyone would love to work in Oppia is mentorship. Ben and Adhiambo are genuinely some of the most responsive and supportive maintainers I have encountered in open source. This kind of mentorship is rare, and spending the whole summer working closely with this team through GSoC is an opportunity I do not want to miss. Also I volunteer with my college club to teach students from remote places that don't have access to good reading material through visuals, hands-on activities, breaking down concepts so that they actually understand, not just memorize. This experience has shown me a lot about how to deliver in education. This is what exactly Oppia does at scale so that's why I want to contribute here not only as a generic open-source project but the mission aligns with something I have been working on.	
What do you hope to learn/achieve during GSoC?	The main thing I want to get better is to design features end to end. So far my contributions have only been fixing the bugs but I have not yet built something new completely yet from the proto layer all the way up to the UI. Project 4.1 would push me to do that exactly. Specially I want to learn how to design proto messages from scratch as I have only changed the pre existing ones, I want to build custom HTML tag handlers, understand how data actually flows from Oppia web backend through export pipelines into Android app and I also want to get better at writing tests as I wrote my first test cases in this repository only and I want to get better at it.	
Which Oppia teams have you collaborated with, and what have you done on those teams?	Android CLaM team	My contributions so far include 4 merged PRs (#6089 , #6101 , #6126 , #6102) and 1 PRs currently in review (#6117).
Contact information	Email: neernbhatta@gmail.com	
Preferred method of communication	Email and Google Chat	

Social media handles (optional) <i>This field is optional. We ask for it so that we can tag you in GSoC-related posts if your application is accepted!</i>	I am on X. You can tag me here link .
Which timezone will you primarily be in during the summer?	(GMT+5:45)
(If you are a student) When are your school holidays?	<i>My College is only scheduled for 4 days a week so I have 3 days for focused work every week. Also, my summer vacation starts early March for 15 days.</i>
What other obligations might you need to work around during the summer?	I don't have any internships, jobs or any other placements planned for this summer. I have my college throughout this period so I also have no travel or other commitments that would affect my presence between May1 - Sept 30.
Planned time commitment	During my summer vacation I plan to work 5-6 hours a day for 6 days a week. Once college resumes I will work for 2-3 hours on college days and 4-5 hours on the rest non-college days.

Project Timeframe

Coding period	I will adhere to the above deadlines.
----------------------	---

Communication Channels

I can commit to sending daily updates to the team group chat, each day I work during the GSoC period.	Yes
In addition to the above: how often, and through which channel(s), do you plan on communicating with your mentor?	I will send daily updates in the team group chat about what I worked on, any blockers and what I plan to do next. For technical and detailed discussion, I will use github discussions and PR threads so that every approach is properly documented. Also we always have a weekly meeting so I plan to attend it regularly.

Contributor Briefing

If accepted, I will be able to attend the contributor briefing at 3 pm UTC on May 1.	Yes
---	---

Section 2: About Your Project

Project Details

Project title <i>(should match the one on the Project Ideas list)</i>	4.1: Support for Lesson Progress, Study Guides, and Worked Examples
Project size <i>(should match the options on the Project Ideas list)</i>	Medium (~175 hours)
Why did you choose this project?	I choose this project because most of the contributions that I have made are in exploration player and the <code>BindableAdapter</code> layer which are the exact system 4.1 builds on. When I was working on the Continue button translation (#6102), I went through <code>StateRetriever</code>, <code>TranslationController</code>, and the customization args parsing flow. The study guide feature needs the same kind of retriever and translation setup. When I was fixing the RecyclerView flash (#6126), I migrated the entire <code>StatePlayerRecyclerViewAssembler</code> to <code>registerViewDataBinder</code> and understood how every item type in the lesson player gets rendered. Lesson progress and worked examples will plug into that same system. When I fixed the audio crash (#6101), I traced the bug through <code>AudioFragmentPresenter</code> and injected <code>NetworkConnectionDebugUtil</code> as a test dependency. <code>StudyGuideFragmentPresenter</code> uses the same presenter pattern, so I already know how to wire dependencies and write tests there. When I worked on #6089, I edited a proto file directly and updated the <code>RetrofitModuleTest</code> assertions to match. Project 4.1 needs new proto messages like <code>CheckpointProgress</code> and <code>StudyGuideSection</code>, I have already done that proto-to-test cycle before. So this felt like a natural next step, not picking a random project, but continuing deeper into the code I already know.

Required Skills (Project-Specific)

GENERAL	
I can figure out the root cause of an issue, and communicate it effectively	Discussion #6093 I needed to figure out how to test a transient UI flash that lasted 50-100ms in the exploration player. I added logging in <code>BindableAdapter</code> and <code>StatePlayerRecyclerViewAssembler</code> to trace the

using a debugging doc .	binding lifecycle and found out that the feedback and submitted answer items go through <code>registerViewBinder</code> not <code>registerViewDataBinder</code> this changed the entire fix approach. The discussion led to the fix of PR #6126 .
I can debug and fix CI failures / flakes.	PR #6089 After I removed the body proto field from the <code>oppia_logger.proto</code>, CI broke because a test was still trying to use that field. I found the failing test (<code>RetrofitModuleTest</code>), removed the assertion that was checking the deleted field, and CI went green. (fix commit) PR #6101 — My changes broke 4 existing <code>AudioFragmentTest</code> cases because they were not setting up network state mocks. CI caught these failures and I fixed them by injecting <code>NetworkConnectionDebugUtil</code> and setting the correct connection status before each test.
I can make changes to GitHub Actions workflows.	I have not modified GitHub Actions workflows yet, but I am familiar with reading CI logs and debugging workflow failures from my PR review cycles.
ANDROID	
Build the app and install it on a local device or emulator. Then verify that you can (a) build a non-test library target locally, (b) run a unit test locally, and (c) play through the app locally.	(a) Here is the Github Gist. (b) Here is the link to StateFragmentTest. (c) Here is the link to a demo Video.
Write new Kotlin code with unit tests.	PR #6101 → Added network aware audio playback in AudioFragmentPresenter.kt with <code>handlePlayPauseButtonClick()</code> that checks connection status before playback. Added <code>playPauseButtonClicked()</code> to LanguageInterface.kt. Wrote 3 new test scenarios in AudioFragmentTest.kt covering: playback when already playing (skips network check), offline dialog when paused without network, and cellular-without-permission dialog. Also fixed 4 existing test cases by properly setting up network state. PR #6126 → Wrote a new helper function <code>synchronizeBindingWithRebind()</code> in BindableAdapter.kt that calls <code>executePendingBindings()</code> to force immediate data binding updates. This was to fix a bug where recycled views briefly showed stale content. Added 2 new unit tests in BindableAdapterTest.kt that verify synchronous binding rebind behavior. Also added a regression test in StateFragmentTest.kt to make sure the old flash does not come back after submitting answers across cards.

Change Android UIs, write tests for them, and manually verify that they work.	PR #6102 → Changed continue_interaction_item.xml from hardcoded <code>@string/state_continue_button</code> to data-bound <code>@{viewModel.buttonText}</code> so the Continue button shows translated custom text from the exploration. Updated ContinueInteractionViewModel.kt with a <code>deriveButtonText()</code> method that uses <code>TranslationController.extractString()</code> for the current language and falls back to default "Continue" if no custom text exists. Tested manually by switching the app to Portuguese and verifying that buttons like "Começar Aventura" showed up correctly. Added tests in StateFragmentTest.kt for both custom text and fallback scenarios. PR #6126 → Switched all item types in StatePlayerRecyclerViewAssembler.kt from <code>registerViewBinder</code> to <code>registerViewDataBinder</code> and added <code>synchronizeBindingWithRebind()</code> helper in BindableAdapter.kt that calls <code>executePendingBindings()</code>. This fixed a UI bug where recycled views were briefly showing previous card content. Verified manually by submitting answers across multiple cards. Added regression test in StateFragmentTest.kt. PR #6117 → Fixed toolbar overlapping the status bar on Android 15+ where edge-to-edge is enforced by default. Added <code>android:fitsSystemWindows="true"</code> to AppBarLayout across around 50 layout files like home_activity.xml, exploration_activity.xml, classroom_list_activity.xml etc. Added inset handling for drawer_fragment.xml and profile chooser screens. Also deleted values-v35/themes.xml entirely because the old opt-out approach was not working with edge-to-edge anymore. Shared screenshots in the PR thread as proof.
---	--

(Optional) Oppia Community References

Who you worked with	Context in which you worked with them
BenHenning	Primary reviewer on PRs #6126 , #6102 , and #6089 . He gave in-depth technical feedback on BindableAdapter and RecyclerView recycling behavior, and answered my questions in Discussion #6088 about how to write tests for the UI flash fix.
adhiamboperes	She reviewed and merged PRs #6101 . On #6117 , she raised multiple issues with the underlying issue itself and gave detailed direction on a different approach, we are currently working through it together in depth.

Section 3: Proposal Details

Problem Statement

Oppia has reached over 2 million learners to date, and its Android app with around 700k installs and 7,000 monthly active users, serves learners who rely on mobile devices as their primary way to access lessons. But the Android app is missing three things that web already has.

First, there is no progress indicator in lessons. A learner playing through an exploration has no clue if they are halfway done or just starting. This is a real problem because many of our users have limited data or share devices with siblings, they need to know if they can finish now or should come back later. PR [#5388](#) tried to add this before but got closed because the checkpoint data wasn't available on Android at the time. This prerequisite has since been resolved, PR [#6185](#) (fixing [#6103](#)) added the `is_checkpoint` field to the `State` proto and updated `StateRetriever` to parse it. The progress visualization itself is what this project will build.

Second, revision cards on Android are just one big wall of text. On web these were already restructured into "study guides" with section headings ([#5844](#)) but Android still shows the old flat format.

Third, the web platform has a worked examples component that shows step-by-step question-answer breakdowns in concept cards. Android has no handler for this tag so all that content is just invisible. Oppia's own user research found that learners who got worked examples after mistakes were able to progress, while those who only got text descriptions got stuck ([PRD](#)). Sweller's cognitive load theory also backs this up, studying worked examples frees up working memory for understanding rather than just problem solving ([Sweller, 1988](#)).

Target Audience	Learners: Ages 7-14, many on Android as their only device, often with limited data. Lesson creators: They write explorations and concept cards on web. The worked examples and study guide content they create needs to actually show up on Android for it to reach learners.
Core User Need	Learners (PRD, Falarini UX study): - Need to see how far they are in a lesson. The UX study found learners are most motivated by seeing their progress and reaching levels. Right now they get nothing. - Need study guides with section headings so they can find specific concepts while revising instead of scrolling through unstructured text. - Need worked examples with clear question-answer format in concept cards. UXR showed that "she was able to progress when given an example but had difficulties with descriptions only" (PRD – Finding 2). Lesson creators: - Need the content they create on the web to actually render on Android. Right now

worked examples are invisible and study guides have no structure.

Section 3.1: WHAT

Key User Stories and Tasks that will be implemented

#	Title	User Story Description (role, goal, motivation) <i>"As a ..., I need ..., so that ..."</i>	List of tasks needed to achieve the goal (this is the "User Journey")	Links to mocks / prototypes, and/or PRD sections that spec out additional requirements.
1	Learner sees progress during a lesson	As a learner, I need to see how far I am in a lesson, so that I can decide whether to continue or come back later.	1. Learner starts an exploration from the topic screen.	No new UI, existing topic screen flow. See #4652
			2. The lesson player loads the first card with a progress indicator next to the navigation button showing "1/N".	Mock 1: Checkpoint 1/5 – Submit , PRD: #6103
			3. Learner answers correctly and moves to the next checkpoint – progress updates (e.g., "2/5").	Mock 2: Checkpoint 4/5 – Correct + Continue
			4. Learner answers incorrectly and gets sent to a concept review card – progress does NOT change since this is not a checkpoint.	Mock 3: Wrong answer – retry at 3/5 Note: progress stays at 3/5, does not increment on non-checkpoint cards.
			5. Learner returns to the main path and continues – progress resumes from where it was.	Same as Mock 2, progress resumes from the last completed checkpoint.
			6. Exploration without any <code>card_is_checkpoint</code> states shows no progress indicator at all.	Mock 6: No Checkpoints – progress hidden
2	Learner views a structured study guide	As a learner, I need study guides with section headings, so that I can find specific concepts while revising instead of	1. Learner opens a topic and taps a subtopic revision card.	No new UI – existing topic screen. See #5844
			2. The app fetches the study guide using the new <code>subtopic_with_study_guide</code>	Backend/domain change – no UI mock. See #5844 comment on endpoint

		scrolling through unstructured text.	endpoint.	
			3. The study guide renders with section headings (bold) and content blocks in a RecyclerView.	Mock 4: Study Guide – structured sections. Web reference: Oppia study guide Figma
			4. When the feature flag is OFF, the old flat revision card format is shown instead.	Existing revision card UI is preserved. Gated by enable_study_guides feature flag.
			5. Learners scroll through structured sections to find the concept they need.	Same as Mock 4 – scrollable RecyclerView with multiple heading/content sections.
3	Learner sees worked examples inline	As a learner, I need to see worked examples with clear question answer format in concept cards, so that I can learn from step by step breakdowns instead of just text descriptions.	1. Learner encounters a concept card or study guide containing an <oppia-noninteractive-workedexample> tag.	Mock 5: Inline Question/Answer. PRD: #5963 , #352
			2. The custom tag handler parses the question and answer HTML.	Internal processing here - no UI changes. See wiki technical guidance on <i>CustomHtmlParser</i> interface.
			3. "Question:" and "Answer:" are rendered inline as bold-labeled blocks with proper spacing.	Same as Mock 5 – shows bold Question: and Answer: labels within content card.
			4. The content works correctly in RTL languages and with translations.	RTL support will be verified during QA. Handled via existing TranslationController and Android RTL layout mirroring.
			5. When the feature flag is OFF, the worked example tag is silently ignored (not rendered).	Content card renders without the worked example block. Gated by enable_worked_examples feature flag.

Testing Plan

Critical user journeys

Feature 1: Lesson Progress

User Type	User Story Description	Test definition		
		Setup (ends with a user of the correct type getting logged in)	Test steps	Test expectations
Learner	Learner sees checkpoint progress at the start of an exploration	- Profile created, logged in. - Test exploration <i>test_exp_id_2</i> updated with <i>card_is_checkpoint</i> set on 3 states. - Feature flag <i>enable_lesson_progress_visualization</i> is ON.	1. Launch exploration activity with <i>test_exp_id_2</i> .	Exploration loads successfully.
			2. Start playing the exploration.	First card content is displayed.
			3. Scroll to the navigation button area.	Progress indicator is visible showing checkpoint 1/N. The progress bar has N circular nodes. First node has "current" state (white with green border), remaining nodes have "incomplete" state (grey border). Content description reads "Checkpoint 1 of N completed".
Learner	Progress updates when learner answers correctly and advances to next checkpoint	Same as above. Exploration loaded and playing at first card.	1. Submit a correct answer on the current card.	Answer is accepted. Feedback card is displayed with positive feedback.
			2. Click "Continue" to advance to the next checkpoint state.	Next card loads with new content.
			3. Scroll to the navigation button area.	Completed checkpoint count increments. Previously visited checkpoint nodes show "done" state (solid green). Current node shows "current" state. Progress fill bar width increases proportionally. Content description updates to "Checkpoint 2 of N completed".
Learner	Progress does NOT change on	Same setup. Learner has advanced to	1. Submit a wrong answer that triggers a flashback to a concept review card.	Flashback card is displayed with review content.

	non-checkpoint or flashback cards	checkpoint 2/N.	2. Scroll to the navigation area on the flashback card.	Progress indicator still shows the same checkpoint count as before the wrong answer. It should not increment or decrement. <code>EphemeralState.flashback_state</code> is true but <code>CheckpointProgress.completed_checkpoint_count</code> is unchanged.
			3. Answer correctly on the flashback card and return to the main path.	Progress resumes from where it was before the flashback. The checkpoint count has not changed.
Learner	Progress indicator hidden for explorations without checkpoints	- Profile created, logged in. - Use an exploration where NO states have <code>card_is_checkpoint</code> set. - Feature flag <code>enable_lesson_progress_visualization</code> is ON.	1. Launch exploration activity and start playing.	Exploration loads and first card content is displayed.
			2. Scroll to the navigation button area.	Progress indicator (checkpoint bar) is NOT visible. Navigation buttons (Submit/Continue) render normally without any progress bar above them. <code>EphemeralState</code> has no <code>checkpoint_progress</code> field set.
Learner	Progress indicator hidden when feature flag is OFF	- Profile created, logged in. <code>test_exp_id_2</code> has <code>card_is_checkpoint</code> set. - Feature flag <code>enable_lesson_progress_visualization</code> is OFF.	1. Launch exploration activity with <code>test_exp_id_2</code> and start playing.	Exploration loads normally.
			2. Scroll to the navigation area.	Progress indicator is NOT visible. <code>EphemeralState.checkpoint_progress</code> is not populated. Layout matches the current production behavior with no progress bar.
Learner	Progress works correctly with backward navigation	Same checkpoint setup. Learner has advanced to checkpoint 3/N.	1. Click the previous button to navigate backward to a completed state before checkpoint 2.	Previous card is displayed as a completed state.
			2. Scroll to the navigation area on the previous state.	Progress indicator shows the checkpoint count corresponding to the state being viewed (e.g., 1/N). Completed checkpoint

				nodes still show "done" state but current indicator has moved backward.
--	--	--	--	---

Feature 2: Study Guide

User Type	User Story Description	Test definition		
		Setup (ends with a user of the correct type getting logged in)	Test steps	Test expectations
Learner	Study guide renders with section headings and content blocks	- Profile created, logged in. <code>test_topic_id_0_1</code> updated with new study guide structure containing multiple sections (heading + content pairs). - Feature flag <code>enable_study_guides</code> is ON.	1. Navigate to the Fractions topic.	Topic page loads with subtopics listed.
			2. Tap a subtopic revision card (e.g., "What Is a Fraction?").	Study guide activity opens with brown toolbar (#BE563C). Toolbar title shows the subtopic name.
			3. Observe the content area.	RecyclerView displays section headings (bold, 18sp) and content blocks (16sp) in the correct order matching the test data. Each section heading text matches the <code>SubtitledUnicode</code> heading. Content renders HTML correctly (formatting, images, links).
Learner	Study guide falls back to old revision card when	- Same test data as above. - Feature flag <code>enable_study_</code>	1. Navigate to the Fractions topic and tap a subtopic revision card.	Old revision card activity opens (not the new study guide activity).

	flag is OFF	guides is OFF.	2. Observe the content.	Content renders as a single flat SubitledHtml block without section headings, matching current production behavior.
Learner	Study guide navigation between subtopics works	- Same setup as above with flag ON. - Topic has multiple subtopics (at least 3).	1. Open the second subtopic's study guide.	Study guide displays with both "Previous" and "Next" navigation cards at the bottom.
			2. Tap the "Next" navigation card.	Study guide for the next subtopic opens with the correct title and section content.
			3. Tap the "Previous" navigation card.	Returns to the previous subtopic's study guide with correct title and content.
Learner	Study guide retains state on configuration change	Same setup. Study guide is open and scrolled to a middle section.	1. Rotate the device (configuration change).	Study guide is still displayed with correct content.
			2. Observe the scroll position and content.	Scroll position is approximately retained. Headings and content blocks are still visible and correct. Toolbar title is preserved.
Learner	Study guide section headings have correct accessibility support	Same setup. Study guide is open.	1. Check the content description of the first section heading item in the RecyclerView.	Content description matches the heading text from the test data.
			2. Verify all section headings have content descriptions.	Each heading item has a content description. TalkBack would announce headings as navigable section elements.

Feature 3: Worked Example

User Type	User Story Description	Test definition
-----------	------------------------	-----------------

		Setup (ends with a user of the correct type getting logged in)	Test steps	Test expectations
Learner	Worked example renders inline with Question/Answer labels	- Profile created, logged in. <code>test_topic_id_0_1</code> updated with an <code><oppia-noninteractive-workedexample></code> tag in concept card / study guide content. - Feature flag <code>enable_worked_examples</code> is ON.	1. Navigate to content containing the worked example tag (e.g., a concept card from the Fractions topic).	Content card loads and displays the worked example inline.
			2. Observe the rendered worked example.	" Question: " label is rendered in bold, followed by the question content on the next line. " Answer: " label is rendered in bold below it, followed by the answer content. Both are inline within the content card with block spacing. No separate colored boxes or collapsible UI.
Learner	Worked example with nested HTML in question/answer fields renders correctly	Same as above. The worked example's <code>question-with-value</code> contains nested HTML (e.g., <code><p></code>, <code><pre></code>, math expressions).	1. Navigate to the content with the worked example containing nested HTML.	Content loads without errors.
			2. Observe the rendered question and answer.	Nested HTML within question and answer is properly parsed into Spannables. Formatting (bold, italic), images, and math expressions render correctly within the inlined worked example. The outer HTML structure is not broken by the nested parsing.
Learner	Worked example tag silently ignored when flag is OFF	- Same test data as above. - Feature flag <code>enable_worked_examples</code> is OFF.	1. Navigate to content containing the worked example tag.	Content card loads normally.
			2. Observe the rendered content.	The <code><oppia-noninteractive-workedexample></code> tag is completely hidden, no Question/Answer labels appear, no error messages, no blank space. Surrounding

				content renders normally as if the tag did not exist.
Learner	Worked example has correct accessibility content description	Same setup with flag ON. Content with worked example is displayed.	1. Check the content description of the rendered text view containing the worked example.	Content description includes "Question:" followed by the question text, then "Answer:" followed by the answer text in logical reading order.
			2. Verify screen reader can read the full worked example.	The full worked example content is accessible. Labels provide structure for assistive technology.
Learner	Worked example renders correctly in RTL layout	- Same setup with flag ON. - Device locale set to an RTL language (e.g., Arabic).	1. Navigate to content containing the worked example.	Content loads correctly.
			2. Observe the layout direction of Question/Answer labels and content.	Content loads correctly.

Technical Requirements

Additions/Changes to the Web Server Interface

#	Endpoint URL	Request type (GET, POST, etc.)	New / Existing	Description of the request/response contract (and, if applicable, how it's different from the previous one). For new endpoints, include which access control decorator (e.g. "can_play_exploration", "open_access", etc.) will be used.
No additions/changes to the Web Server interface are required for this project.				

New/Changed Calls by the Web Frontend /Android Client to the Web Server Interface

#	Endpoint URL	Request type (GET, POST, etc.)	Description of why the new call is needed, or why the changes to an existing call are needed
No new or changed calls to the Web Server interface are required. All data (explorations, study guides)			

is loaded from local assets bundled with the app.

Web Server Storage Model Additions/Changes

#	Datastore model	Description of changes	How will existing data be handled?
1.	No web server storage model changes are required.		

UI Screens/Components

#	ID	Description of new UI component	i18n required?	Mock/spec links	What A11y support will be implemented?
1.	LessonProgressIndicator	Horizontal checkpoint progress bar with circular nodes (completed / in-progress / incomplete states). Added to continue_interaction_item.xml , continue_navigation_button_item.xml , and next_button_item.xml layouts. Controlled via StatePlayerRecyclerViewAssembler . Hidden when the exploration has no card_is_checkpoint states or when the feature flag is OFF. Only enabled for explorations, not questions.	Yes, heading "Learning Progress" and the numbered node progress fraction (e.g. "1/5") need to be translatable.	Figma Mocks 1-3 and 6: Figma link	TalkBack : "Learning Progress, X of N completed". Progress bar is a single focusable element, nodes are not individually focusable.
2.	Study Guide Screen	New app/topic/studyguide package (largely copies from app/topic/revisioncard). Uses a RecyclerView with BindableAdapter supporting two item types: section heading (bold, 18sp) and section content (16sp, HTML-rendered). Brown toolbar (#BE563C) matching the existing revision card pattern. Replaces the flat revision card view when the enable_study_guides feature flag is ON.	Yes, section headings are Subtitled Unicode and content is Subtitled Html , both supporting translations via TranslationController .	Figma Mock 4: Figma link . Web reference: Oppia study guide Figma	Section headings announced as headers by TalkBack. Content is fully readable. Navigation cards have content descriptions (e.g., "Previous: What Is a Fraction?", "Next: Equivalent Fractions").
3.	Worked Examples UI	New CustomTagHandler in HtmlParser for the <oppia-noninteractive-	Yes, question and answer text are Subtitled	Figma Mock 5: Figma link	Content description includes

		workedexample > tag. Inlines "Question:" and "Answer:" as bold-labeled blocks with block spacing within existing content cards. Parses nested HTML in the question-with-value and answer-with-value attributes via a new CustomHtmlParser interface added to CustomTagHandler.handleTag() . When the enable_worked_examples flag is OFF, a no-op handler silently ignores the tag.	Html and support translations via TranslationController . "Question:" and "Answer:" labels need translatable strings. Works with RTL layouts (Arabic).		"Question:" followed by question text, then "Answer:" followed by answer text via handleTagForContentDescription() . Full content readable by screen reader in logical order.
--	--	--	--	--	---

Data Handling and Privacy

#	Type of data	Description	Why do we need to store this data?	Anonymized?	Can the user opt out?	Wipeout policy	Takeout policy
1.		No new user data is collected or stored by this project. Lesson progress (the completed state-checkpoint count) is computed ephemerally from the exploration state graph at runtime and is not persisted beyond the existing exploration checkpoint mechanism (ExplorationCheckpointDatabase). Study guide content and worked examples are read-only content loaded from local assets. No analytics events beyond the existing exploration event logging are introduced.					

Connection to Existing Work

Lesson Progress Indicator:

PR [#5388](#) was a prior attempt at adding lesson progress visualization. It was closed because the checkpoint data wasn't available on Android. That prerequisite has now been resolved PR [#6185](#) added [bool is_checkpoint = 10](#) to the [State](#) proto and updated [StateRetriever](#) to parse it from JSON. Test explorations ([test_exp_id_2](#)) already have checkpoint states marked. This project builds the full progress visualization pipeline on top of this existing infrastructure. This project resumes that work.

The exploration player already renders different types of items like content, feedback, submitted answers and navigation buttons through [StatePlayerRecyclerViewAssembler](#). Each item type follows the same pattern: a [StateItemViewModel](#) subclass, a layout XML file and a [registerViewDataBinder](#) call in the assembler. The new progress indicator will follow the same pattern and plug into the existing assembler. [StateDeck](#) already tracks state transitions during a lesson so I will build on top of it to count completed checkpoints.

Study Guide:

The app already has a full revision card feature. [RevisionCardRetriever](#) loads subtopic content from local assets using [topicId](#) and [subtopicId](#), and [RevisionCardFragment](#) renders the HTML content through [HtmlParser](#). [RevisionCardActivity](#) handles the navigation and toolbar. The new [StudyGuideRetriever](#) and [StudyGuideFragment](#) will follow this same retriever-fragment-activity pattern. The only difference is that

instead of reading a single *page_contents* block, the study guide reads a sections list with headings and content pairs.

Worked Example Tag Handler:

On the web platform, worked examples were earlier stored as a separate *worked_examples* list on the skill model, but that was removed in GSoC 2025 ([oppia/oppia#19851](#)) and replaced with a rich text component. Now worked examples are written directly inside the concept card explanation HTML using the tag `<oppia-noninteractive-workedexample question-with-value="..." answer-with-value="...">`. So the data is not coming from a proto field anymore, it is embedded inside the HTML string itself.

On Android, the ConceptCard message in [topic.proto](#) does have an old *repeated SubtitledHtml worked_example = 4* field, and ConceptCardRetriever still fills it from skills.json, but it is never actually used. No UI code in app/src/main reads this field, and there is also a comment in ConceptCardRetriever which says the question part of worked examples is not supported by the app. So this field is basically a dead leftover from the old schema and I will not touch it or change the proto.

The actual implementation happens fully on the HTML rendering side. HtmlParser already registers many custom tag handlers through CustomHtmlContentHandler, for example ImageTagHandler for *oppia-noninteractive-image*, MathTagHandler for *oppia-noninteractive-math*, ConceptCardTagHandler for *oppia-noninteractive-skillreview* and so on. All of them implement the same CustomTagHandler interface. The new WorkedExampleTagHandler will implement this same interface and get registered in HtmlParser the same way, so that whenever the *oppia-noninteractive-workedexample* tag appears inside any rich text content it gets handled properly the same way.

Other Requirements

Feature flags:

All three features must be independently gated behind feature flags (*enable_lesson_progress_visualization*, *enable_study_guides*, *enable_worked_examples*). Each feature must be completely invisible when its flag is OFF, with no behavioral changes to existing functionality.

Backward compatibility:

When the study guide flag is OFF, the existing *RevisionCardActivity* and *RevisionCardFragment* must continue to work unchanged. The old *SubtopicRecord* format must still be parseable.

Launch readiness:

Per the project description, all three features must be fully launched by the end of Milestone 2. This includes 2 weeks for QA testing, bug fixes, and PM reviews scheduled within each milestone.

Section 3.2: HOW

Existing Status Quo

The Oppia Android app currently has three gaps compared to the web platform:

1. **No lesson progress visualization.** When a learner plays an exploration, the UI shows the current card's content, interaction, and navigation buttons, but gives no indication of how far they are through the lesson. The [StateDeck](#) tracks visited states internally, and [ExplorationProgressController](#) builds *EphemeralState* objects from it, but none of this progress data is exposed to the UI layer. The *State* proto now includes an *is_checkpoint* field (added in PR [#6185](#)), and [StateRetriever](#) already parses it from both proto and JSON. However, no part of the domain or UI layer uses this field yet no checkpoint counting, no *CheckpointProgress* proto, and no progress visualization exists. A prior attempt (PR [#5388](#)) was closed before the field existed; now that the data is available, this project will build the full progress tracking pipeline on top of it.
2. **Flat revision cards instead of structured study guides.** Subtopic content is loaded via [RevisionCardRetriever.loadRevisionCard\(\)](#), which reads a *SubtopicRecord* proto from local assets. This proto has a single *page_contents* field (*SubtitledHtml*) one monolithic block of HTML text. The web team (GSoC 2025, issue [#5844](#)) already restructured subtopic pages into study guides with heading/content section pairs and created the *ACTIVITY_TYPE_SUBTOPIC_WITH_STUDY_GUIDE* export endpoint in [android.py](#). Android still consumes the old flat format.
3. **No worked example rendering.** On the web platform, worked examples are now shown using a rich text component, `<oppia-noninteractive-workedexample question-with-value="..." answer-with-value="...">`, which is embedded directly inside the concept card explanation HTML. The earlier *worked_examples* list field on the skill model was removed in GSoC 2025 ([oppia/oppia#19851](#)), and the new RTE component is the only path now. On Android, [HtmlParser.computeCustomTagHandlers\(\)](#) does not register a handler for this tag at the moment, so when the tag appears inside concept card HTML it just passes through the parser as an unknown element and gets silently dropped during rendering.

Note: the ConceptCard message in [topic.proto](#) still has an old *repeated SubtitledHtml worked_example = 4* field, and ConceptCardRetriever still fills it from skills.json. But no UI code reads it, and only the explanation part of each worked example is even extracted, there is a comment inside ConceptCardRetriever which says the question part was never supported by the app. So this field is basically a legacy leftover and not the source of truth for the new feature. Cleaning it up is out of scope for this project.

Solution Overview

The solution adds three independently feature-flagged capabilities to the Android app:

Lesson Progress Indicator

The State proto already has a *bool is_checkpoint = 10* field (merged in PR [#6185](#)), and [StateRetriever](#) already parses it from the exploration JSON. What's missing is the domain logic and UI to use this data. On initialization, [StateGraph](#) performs a pathfind from the initial state to the single terminal state following the answers marked with [labelled as correct](#) and counts how many states on that path have *is_checkpoint = true* to get the totalCheckpointCount (plus 2, for the initial and terminal states). This value is lazy-initialized and memoized. [StateDeck](#) then tracks the learner's completed checkpoint count as they move through states.

The progress count is not a simple incrementing counter. It is computed based on the current *stateIndex* in the deck specifically, how many checkpoint states have been completed up to the state being viewed. This means:

- When the learner navigates backward using [navigateToPreviousState\(\)](#), *stateIndex* decreases, and the displayed progress also goes down. For example if the learner has completed checkpoints 1, 2, 3 and navigates back to a state before checkpoint 2, the bar shows 1/N not 3/N.
- When the learner navigates forward again using [navigateToNextState\(\)](#), the progress goes back up.
- During a flashback (*EphemeralState.flashback_state = true*), the progress indicator shows the checkpoint count of the flashback state being reviewed, not the latest state. When the learner returns from the flashback via [moveBackToLatest\(\)](#), the progress resumes from where they were before.

For resuming a lesson, the app already has a separate "exploration checkpoint" mechanism as [ExplorationCheckpointController](#) saves the exploration state (pending state name, completed answers, state index) into an ExplorationCheckpoint proto via [StateDeck.createExplorationCheckpoint\(\)](#) so learners can resume later. The completed state-checkpoint count will be saved alongside this existing data so that when an exploration checkpoint is restored, the progress bar picks up from where the learner left off.

[View full diagram](#)

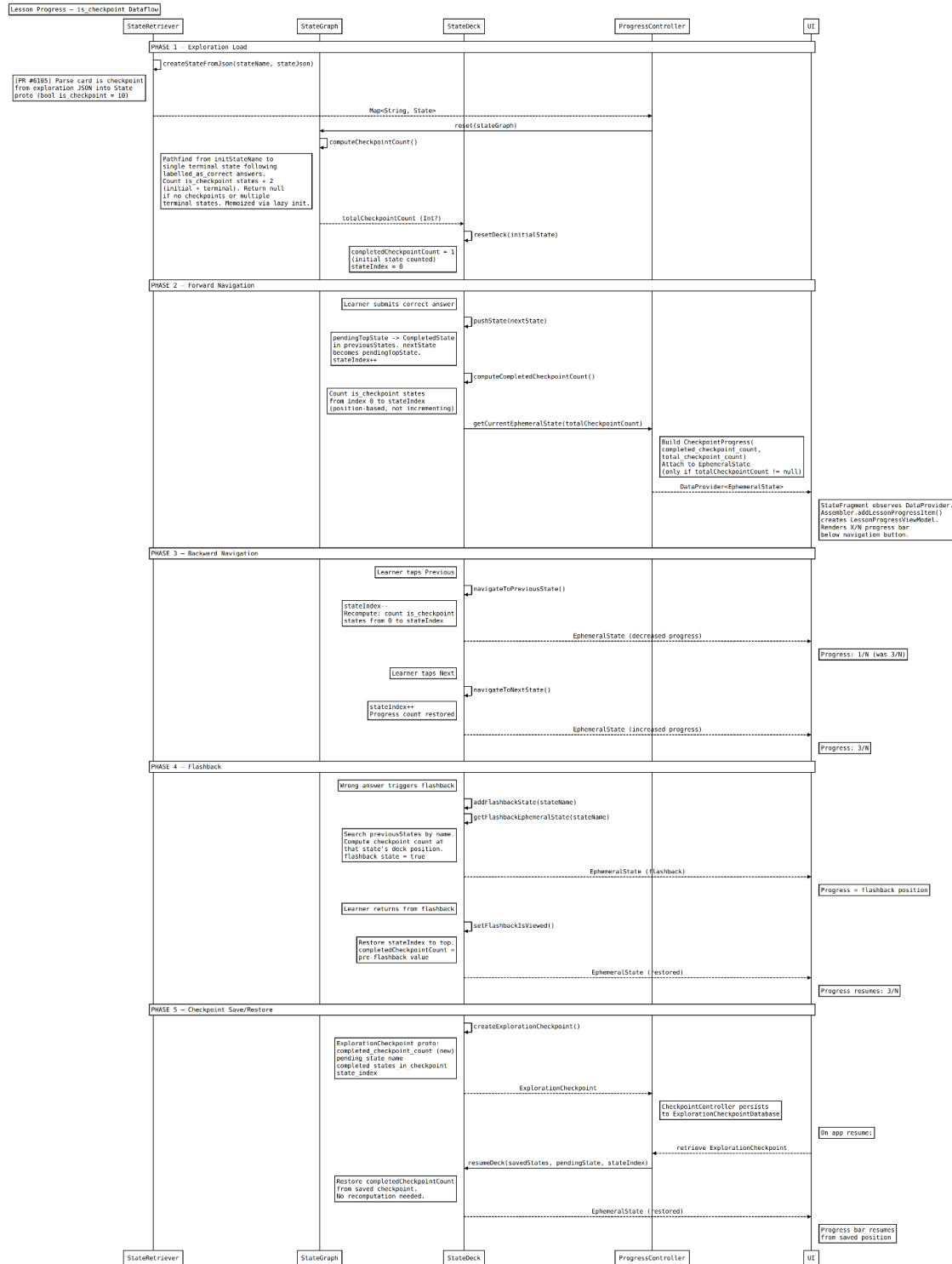
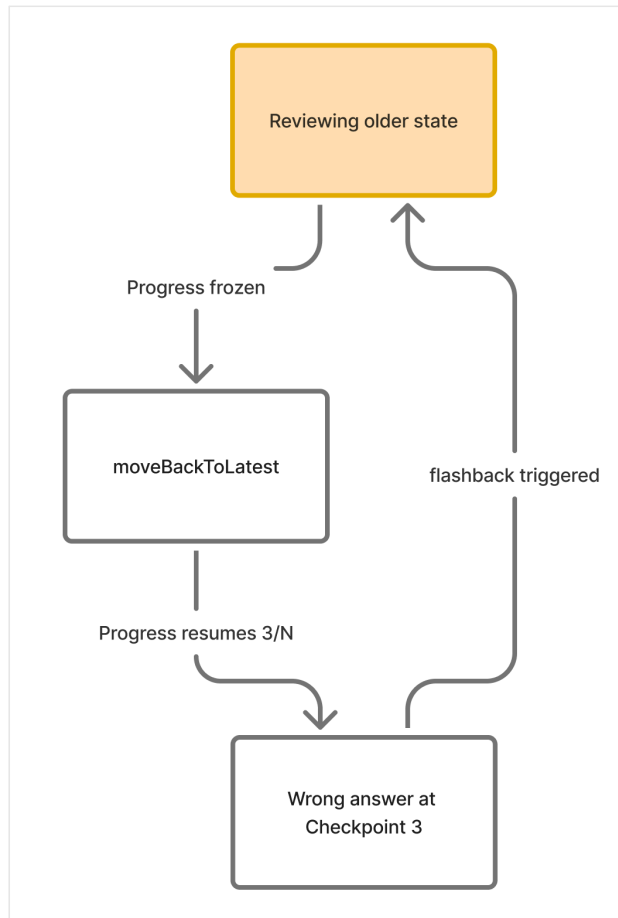
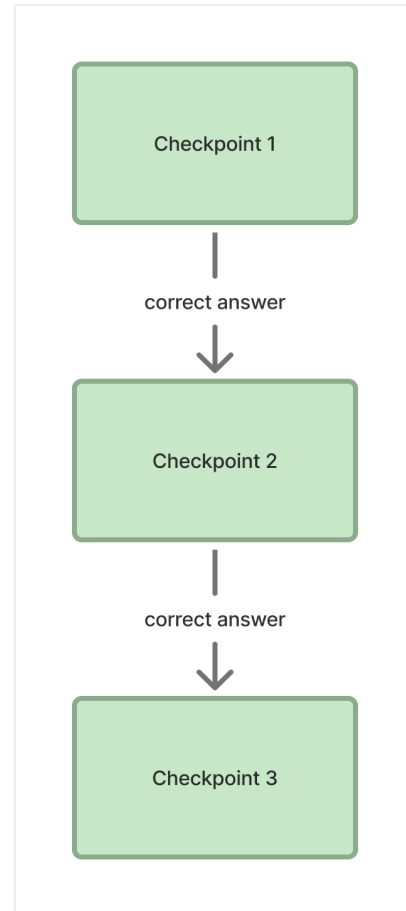


Figure 1: Sequence diagram showing *is_checkpoint* dataflow. [View full diagram](#)

Flashback



Forward Navigation



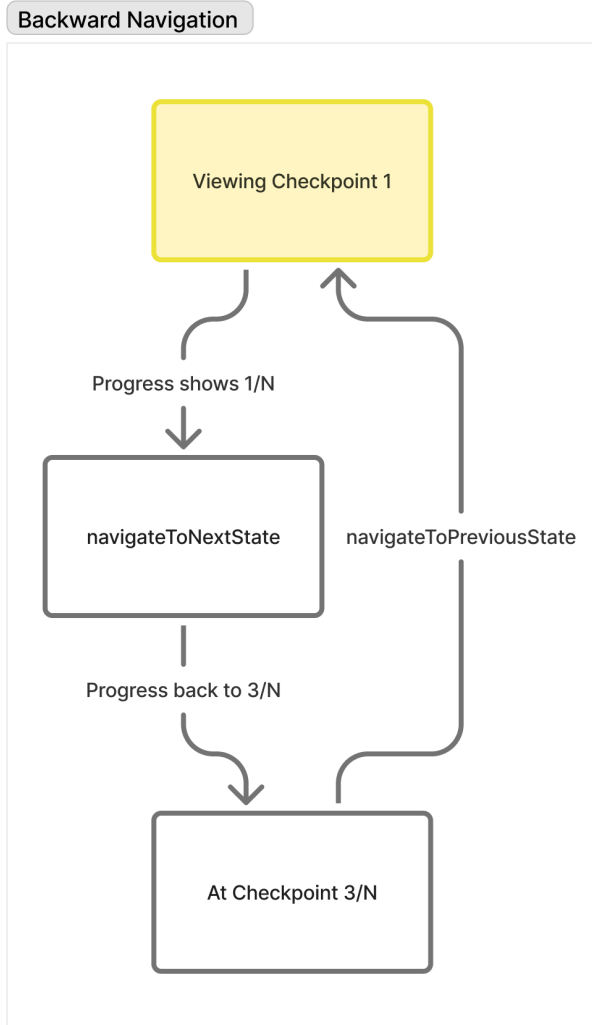


Figure 2: How progress count behaves during forward navigation, backward navigation, and flashback.

Study Guide

The new *StudyGuideFragment* renders the study guide content using a RecyclerView with BindableAdapter, following the same multi-view-type pattern used in StatePlayerRecyclerViewAssembler.

The adapter has two item types registered via registerViewDataBinder:

- **SECTION_HEADING** – a bold text item for section titles
- **SECTION_CONTENT** – an HTML-rendered TextView for section content, parsed through *HtmlParser.parseOppiaHtml()* so images, math and concept card links all work

The existing [RevisionCardFragment](#) renders the entire revision card as a single HTML block in one [TextView](#). The study guide is different because the data has multiple sections, so a [RecyclerView](#) is needed instead.

Translations are handled the same way as the existing revision card, [TranslationController.extractString\(\)](#) extracts the translated HTML string for each section's [SubtitledHtml](#) before passing it to [HtmlParser](#). The [WrittenTranslationContext](#) is attached to the [EphemeralStudyGuide](#) so that each heading and content block can be independently translated.

Worked Example Tag Handler

The main challenge here is that the [question-with-value](#) and [answer-with-value](#) attributes inside the [oppia-noninteractive-workedexample](#) tag can themselves contain HTML (bold text, images, math expressions etc.).

The handler extracts these attribute values using [getJsonStringValue\(\)](#) (the same JSON extraction pattern used by [ConceptCardTagHandler](#) to extract the [skillId](#) and text attributes). For the inner HTML content, [CustomHtmlContentHandler.CustomTagHandler.handleTag\(\)](#) and [handleTagForContentDescription\(\)](#) are updated to accept a new CustomHtmlParser interface that takes an HTML [String](#) and returns a [Spannable](#). A default implementation of this interface is provided via [CustomHtmlContentHandler.fromHtml\(\)](#). [WorkedExampleTagHandler](#) uses this [CustomHtmlParser](#) to parse the inner question/answer HTML, which properly handles nested formatting (bold, italic, images, math) without re-invoking the full tag handler pipeline and risking infinite recursion.

The handler then builds a [SpannableStringBuilder](#) with:

1. A bold "Question:" label ([StyleSpan\(Typeface.BOLD\)](#)) + the parsed question [CharSequence](#)
2. A line break
3. A bold "Answer:" label + the parsed answer [CharSequence](#)
4. [LeadingMarginSpan](#) for block indentation

This entire spannable replaces the tag's content in the outer [Editable](#) using [output.replace\(openIndex, closeIndex, spannableBuilder\)](#), the same pattern used by [ImageTagHandler](#) and [ConceptCardTagHandler](#).

For translations the HTML content inside the tag attributes is already translated by the web export pipeline before it reaches Android. The "Question:" and "Answer:" labels are Android string resources ([R.string](#)) that get translated through the app's locale system.

For RTL the [SpannableStringBuilder](#) respects the parent [TextView's layoutDirection](#) which Android sets automatically based on the app locale. The handler does not hardcode any directional formatting so it works for Arabic, Hebrew etc. The [handleTagForContentDescription\(\)](#) method provides the content in logical reading order (question first, then answer) for screen readers.

One thing worth calling out is that the attribute value still has one layer of HTML escaping in it after [getJsonStringValue](#) strips the outer quotes. The source attribute is double-encoded in the HTML and Android's SAX parser only undoes one layer when it reads attributes. So before I pass it into the recursive

parser, I unescape entities one more time with a small helper, otherwise the parser will see literal `<p>` text instead of a paragraph tag.

To make sure the recursive call never infinite-loops, the inner handler map I pass to `fromHtml` is built without the worked-example key. So the recursion is safe by construction, not by convention.

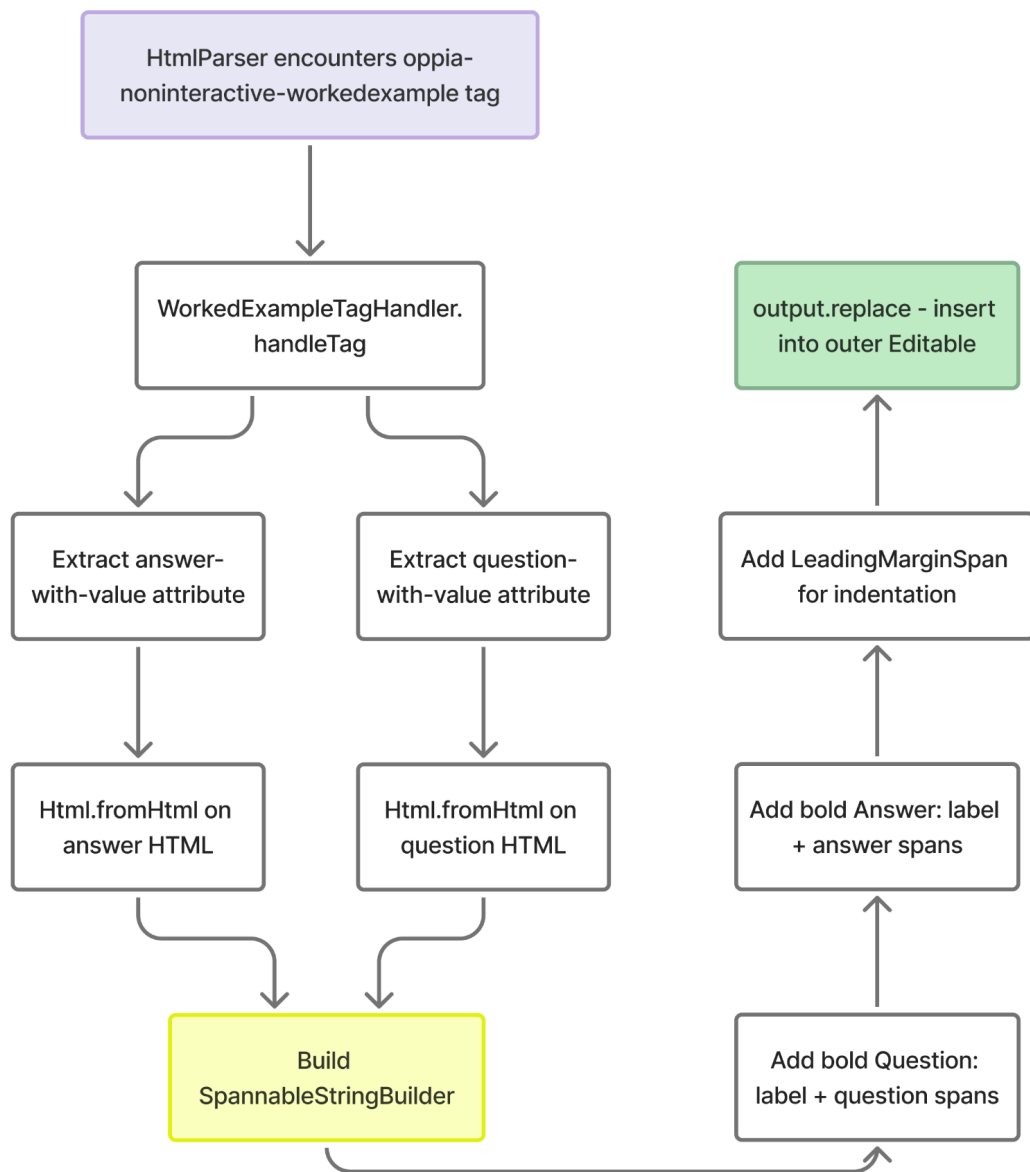


Figure 3: How *WorkedExampleTagHandler* processes nested HTML and builds spannables.

Third-Party Libraries

No.	Third-party library name and version	Link to third-party library	Why it is needed	License (if third-party library)	[Android only] Min / target / max SDK version that the library supports
1	No new third-party libraries are required. All three features use existing infrastructure: Protocol Buffers for data models, Android DataBinding for UI, and the custom HTML parsing pipeline (CustomHtmlContentHandler) for worked examples.				

“Service” Dependencies

No.	Dependency name	Why it is needed	What our plan is, if the dependency fails under us
1	No new service dependencies are introduced. The app continues to load all content from local proto/JSON assets. The is_checkpoint field prerequisite has been resolved (PR #6185 , closing #6103). For study guides, #6105 (study guide support in the lesson export pipeline) is expected to be resolved before the GSoC coding period begins. The study guide proto structure in this proposal may need minor adjustments based on the final export format from #6105 .		

Key High-Level and Architectural Decisions

Decision 1: How to compute lesson progress, checkpoint-based vs. total-state-based

We have considered the following alternatives:

1. **Total-state count:** Progress = visited states / total states in the state graph. Simple, but misleading — branching paths and wrong-answer loops mean the learner may visit more states than exist on the "happy path," making progress appear to go backward or exceed 100%.
2. **Checkpoint-based count:** Progress = completed checkpoints / total checkpoints. Checkpoints are curated by lesson creators (via [card_is_checkpoint](#)) to mark meaningful milestones. This gives a monotonically increasing progress metric that matches the learner's mental model.
3. **Pathfinding-based (Dijkstra):** Follow [labelled_as_correct](#) answers from the initial to the terminal state to find the main learning path, then count checkpoints on that path. Gives an accurate count but adds implementation complexity.

Among these, a **combination of (b) and (c)** is the best approach: **checkpoint-based counting on the main path found via pathfinding**. Specifically, [StateGraph](#) uses Dijkstra's algorithm once on exploration load to find the main path (following [labelled_as_correct](#) answers) and counts checkpoints on that path plus 2 (for initial and terminal states). This value is memoized. Then at runtime, completed checkpoint counting is O(1) per state transition — [StateDeck](#) computes how many checkpoint states up to the current [stateIndex](#) have been completed.

- It only counts checkpoints on the main learning path (not branching/wrong-answer paths), ensuring accuracy.

- The pathfinding runs once per exploration load and is memoized, so the per-transition cost remains $O(1)$.
- It gracefully handles explorations without checkpoints: returns *null*, and the progress bar is hidden.
- It is the approach suggested in the project's [technical guidance](#).

The above approaches are contrasted in detail in the following table:

Criterion	(a) Total-state count	(b) Checkpoint-based	(c) Dijkstra pathfinding
Monotonically increasing	No – branching causes regression	Yes	Yes (on happy path)
Computation cost	$O(1)$	$O(1)$	$O(V + E)$ per state
Handles branching	Poorly	Well – checkpoints are on main path	Assumes single path exists
Web platform parity	No	Yes	No
Requires new data	No	Yes, <i>is_checkpoint</i> (already available)	No

Decision 2: Study guide architecture – new fragment vs. extending RevisionCardFragment

We have considered the following alternatives:

1. **Extend RevisionCardFragment:** Add conditional logic to render either flat content or sectioned content depending on a flag. Reuses existing code but adds complexity to an already-working component and risks regressions.
2. **New StudyGuideFragment:** Create a parallel fragment/activity/presenter set under *app/topic/studyguide/*, copying the structural pattern from *revisioncard/* but with a *RecyclerView* for sections instead of a single HTML view. Clean separation of concerns.

Among these, **option (b) new fragment** is the best approach, because:

- It follows Oppia Android's established pattern of feature-flagged components being separate entities (e.g., how flashback support was added as new view models rather than conditionally extending existing ones).
- When the feature flag is OFF, the existing *RevisionCardFragment* continues to work with zero code path changes – there is no risk of breaking the existing revision card experience.
- The routing logic is a single conditional in *TopicRevisionFragmentPresenter*: if study guides are enabled, launch *StudyGuideActivity*; otherwise, launch *RevisionCardActivity*.

Decision 3: Worked example rendering – inline SpannableString vs. custom View

We have considered the following alternatives:

1. **Custom View inflated into content:** Create a `WorkedExampleView` with its own layout XML, inflated and inserted into the content area. Provides pixel-perfect control but breaks the flow of the surrounding HTML text and requires managing view lifecycle within a `TextView`.
2. **Inline SpannableString rendering:** Render question/answer content as styled spans within the existing `TextView`, using bold `StyleSpan` for labels and `LeadingMarginSpan` for block formatting. Keeps the content flowing naturally within the text and follows the same pattern as `ImageTagHandler` and `ConceptCardTagHandler`.

Among these, **option (b) inline SpannableString** is the best approach, because:

- It uses the same mechanism as every other custom tag handler in the codebase – no new rendering paradigm is introduced.
- It naturally supports accessibility: the `handleTagForContentDescription()` method provides a linearized reading order for TalkBack.
- It handles nested HTML (bold, italic, links within question/answer text) by delegating to a new `CustomHtmlParser` interface that re-invokes the parser for inner HTML content.
- The GSoC project description specifies that Android should render worked examples as bold "Question:"/"Answer:" labels followed by content, matching how lesson authors previously represented this before web moved to its current richer collapsible UI. This maps directly to `StyleSpan(BOLD)` + plain text spans.

Risks and mitigations

Potential Risk	Mitigation
Nested HTML parsing in worked examples could cause infinite recursion or malformed spans if the inner question/answer content contains other custom Oppia tags.	<code>WorkedExampleTagHandler</code> will use a new <code>CustomHtmlParser</code> interface (default implementation via <code>CustomHtmlContentHandler.fromHtml()</code>) for inner content rather than re-invoking the full <code>CustomHtmlContentHandler</code> pipeline. This is necessary because the inner question/answer HTML can contain other custom tags (e.g. <code>oppia-noninteractive-image</code> , <code>oppia-noninteractive-math</code> , <code>oppia-noninteractive-skillreview</code>) that must be rendered correctly. Excluding only the worked example handler from the inner parse prevents infinite recursion while still supporting all other nested rich text components. Edge cases covered in unit tests: empty question/answer, malformed JSON-encoded attribute, double-escaped HTML entities, and nested image / math / concept-card inside the question text.

Study guide proto structure may not correctly represent the sectioned format (headings + content pairs), leading to missing or misrendered sections.	The lesson download script (issue #6105) handles the translation from web's JSON export to Android's proto format, so the app only needs to consume the proto correctly. <i>StudyGuideRetriever</i> will be tested against proto test assets with multiple sections, empty sections, and single-section edge cases to verify correct parsing. Integration tests will load these test protos and assert that all headings and content blocks render correctly.
Progress count could show incorrect values during backward navigation or flashback if the checkpoint counting logic has off-by-one errors. This would confuse learners about where they are in the lesson.	The completed count is computed dynamically from stateIndex, not a separate counter, so there is one source of truth. Unit tests will cover every navigation path: forward through checkpoints, backward past checkpoints, flashback and return, and edge cases like navigating back to the very first state.
Routing to StudyGuideActivity when the flag is ON could break access to existing revision cards if the conditional in <i>TopicRevisionFragmentPresenter</i> has a bug.	When the flag is OFF, the routing logic is completely unchanged and <i>RevisionCardActivity</i> launches as before. A dedicated flag-off test will verify that the existing revision card flow is untouched. The routing is a single conditional check, keeping the risk surface minimal.

Key Objectives for Each Milestone

Milestone	Key Objective
Milestone 1	Lesson progress indicator and study guide both fully implemented end-to-end (proto → domain → UI). All code behind feature flags. QA testing completed for both features with bugs triaged (due Jul 10).
Milestone 2	Worked example tag handler complete. Bug fixes for all three features. All three feature flags turned ON and launched. QA testing + PM review before final submission (due Sep 2).

Implementation Approach

Domain Objects

Feature 1: Lesson Progress

The State proto already has a [bool is_checkpoint = 10](#) field (added in PR [#6185](#)) that distinguishes checkpoint states. The remaining changes needed are:

- A new *CheckpointProgress* message to hold the learner's current progress through checkpoint states.
- A new *checkpoint_progress* field on [EphemeralState](#) so the UI layer can read the progress.

New proto messages in [exploration.proto](#):

```
// Added to State message:
bool is_checkpoint = 10;

// New message:
message CheckpointProgress {
  int32 completed_checkpoint_count = 1;
  int32 total_checkpoint_count = 2;
}

// Added to EphemeralState:
CheckpointProgress checkpoint_progress = 13;
```

[StateDeck](#)

- Add a helper method *computeCompletedCheckpointCount(): Int* that counts how many states with *is_checkpoint = true* have been completed up to the current *stateIndex*. This count is position-based, not an incrementing counter, so:
 - When the learner navigates backward via [navigateToPreviousState\(\)](#), *stateIndex* decreases and the completed count decreases accordingly (e.g., back from 3/N to 1/N).
 - When the learner navigates forward via [navigateToNextState\(\)](#), the count increases back.
 - During a flashback (*flashback_state = true*), the count reflects the flashback state's position, not the latest state.
- Update [getCurrentEphemeralState\(\)](#) to accept a new nullable parameter *totalCheckpointCount: Int?* passed from [ExplorationProgressController](#). When this parameter is *null* (feature disabled OR exploration has no checkpoints), the *checkpoint_progress* field is not set in the returned [EphemeralState](#).
- When *totalCheckpointCount* is non-null, populate *CheckpointProgress* with *completed_checkpoint_count* from the helper above and *total_checkpoint_count* from the parameter.

StateGraph

- Add a new constructor parameter `initStateName: String` so the graph knows the starting state.
- Add a new lazy-initialized property `checkpointCount: Int?` backed by `computeCheckpointCount(): Int?`.
- `computeCheckpointCount()` implements the following:
 - Identifies all terminal states (states with no outgoing transitions via [Outcome.dest_state_name](#)). If there is not exactly one terminal state, logs a warning via `oppiaLogger` and returns `null`.
 - Performs a pathfind from `initStateName` to the single terminal state using Dijkstra's algorithm: for each state, follows the [AnswerGroup](#) whose [Outcome](#) has `labelled_as_correct = true`.
 - Walks the discovered path and counts all states with `is_checkpoint = true`.
 - Returns that count + 2 (to include the initial and terminal states), or `null` if zero `is_checkpoint` states exist on the path.
- This runs once per exploration load and is memoized via `lazy` initialization.
- Tests will cover: single path, multiple correct answers converging, multiple terminal states (returns null), and zero checkpoints (returns null).

Feature 2: Study Guide

The existing [RevisionCardRetriever](#) loads a single `SubtopicRecord` with one `page_contents` block. The study guide format uses multiple sections instead. The following changes are needed:

- New proto messages to represent the sectioned study guide format.
- A new `StudyGuideRetriever` class following the same pattern as `RevisionCardRetriever`.
- A new `getStudyGuide()` method on [TopicController](#) to expose the data.

Note: [#6105](#) (study guide support in the lesson export pipeline) is being addressed by @BenHenning and is expected to be resolved before the GSoC coding period begins. The proto structure below may require minor adjustments to align with the final export format, but the overall architecture (`StudyGuideRetriever` → `StudyGuide` → `EphemeralStudyGuide` → UI) will remain the same.

New proto messages in [topic.proto](#):

```

message StudyGuideSection {
  SubtitledUnicode heading = 1;
  SubtitledHtml content = 2;
}

message StudyGuideRecord {
  SubtitledHtml title = 1;
  repeated StudyGuideSection sections = 2;
  map<string, VoiceoverMapping> recorded_voiceover = 3;
  map<string, TranslationMapping> written_translation = 4;
  repeated string skill_ids = 5;
  LessonThumbnail subtopic_thumbnail = 6;
}

message StudyGuide {
  SubtitledHtml subtopic_title = 1;
  repeated StudyGuideSection sections = 2;
  map<string, VoiceoverMapping> recorded_voiceovers = 3;
  map<string, TranslationMapping> written_translations = 4;
}

message EphemeralStudyGuide {
  StudyGuide study_guide = 1;
  WrittenTranslationContext written_translation_context = 2;
}

```

StudyGuideRetriever

- New class following the [RevisionCardRetriever](#) pattern.
- Loads StudyGuideRecord from local assets via `assetRepository.loadProtoFromLocalAssets("${topicId}_${subtopicId}", StudyGuideRecord.getDefaultInstance())`.
- Converts it to a *StudyGuide* domain object.
- Also supports JSON fallback via *jsonAssetRetriever*.

Feature 3: Worked Examples

No new domain objects needed. The worked example content is embedded in HTML via the `<oppia-noninteractive-workedexample>` RTE tag, which can appear in any rich text content (concept cards,

study guides, and potentially other areas). The new tag handler will be registered globally in *HtmlParser* and will parse the tag wherever it appears at the UI layer.

User Flows (Controllers and Services)

Feature 1: Lesson Progress

Data flow: [Exploration.states\[\]](#).is_checkpoint → [StateGraph.computeTotalCheckpointCount\(\)](#) → [StateDeck](#) (tracks completed checkpoints as states are pushed) → [EphemeralState.checkpoint_progress](#) → [StatePlayerRecyclerViewAssembler.compute\(\)](#) → [LessonProgressViewModel](#) → UI.

The [ExplorationProgressController](#) passes the state graph to [StateDeck](#) on initialization. [StateDeck](#) computes the completed checkpoint count dynamically based on the current *stateIndex* position, not an incrementing counter. This ensures correct counts during backward/forward navigation and flashbacks. [getCurrentEphemeralState\(\)](#) includes the *CheckpointProgress* in the returned proto. No changes to the controller's public API — the data flows through the existing [getCurrentState\(\)](#): [DataProvider<EphemeralState>](#) observable.

Feature 2: Study Guide

Data flow: [TopicController.getStudyGuide\(topicId, subtopicId\)](#) → [StudyGuideRetriever.loadStudyGuide\(\)](#) → *StudyGuide* proto → [TranslationController](#) wrapping → [EphemeralStudyGuide](#) → [StudyGuideViewModel](#) → UI.

[TopicController](#) gets a new [getStudyGuide\(\)](#) method (parallel to existing [getRevisionCard\(\)](#)). The controller wraps the result in an [EphemeralStudyGuide](#) with the appropriate *WrittenTranslationContext* for the user's selected language.

Feature 3: Worked Examples

No controller changes. The tag handler operates at the HTML parsing layer — it processes `<oppia-noninteractive-workedexample>` tags inline when [HtmlParser.parseOppiaHtml\(\)](#) is called on concept card content. The handler reads *question-with-value* and *answer-with-value* attributes, parses the inner HTML, and renders styled spans.

[Android only] UI changes

Feature 1: Lesson Progress Indicator

- **New *ViewType.LESSON_PROGRESS_INDICATOR*** added to [StateItemViewModel.ViewType](#) enum.
- **New *LessonProgressViewModel***: Extends [StateItemViewModel](#). Holds *completedCount: Int*, *totalCount: Int*, and *progressText: String* ("X of N"). The progress bar width is determined by the layout using these raw counts via data binding, following the same pattern as [SegmentedCircularProgressView](#) which receives raw chapter counts rather than pre-computed fractions.

- **New layout `lesson_progress_indicator_item.xml`:** Horizontal bar with circular checkpoint nodes. Uses a custom drawable with three states: completed (filled circle), in-progress (half-filled), incomplete (empty circle). Content description: "Learning Progress, X of N completed."
- **`StatePlayerRecyclerViewAssembler` changes:** New `addLessonProgressItem()` method called from `compute()` when the state type is `PENDING_STATE` or `COMPLETED_STATE` and the exploration has checkpoints at least one state checkpoint. The progress item is added to `extraInteractionPendingItemList` (after the continue/next button). Guarded by `PlayerFeatureSet.lessonProgressSupport` (new field) and the `enable_lesson_progress_visualization` feature flag.
- **Modified layouts:** `continue_interaction_item.xml`, `continue_navigation_button_item.xml`, and `next_button_item.xml`, the progress indicator is rendered below these buttons. I have direct experience modifying `continue_interaction_item.xml` (PR #6102) and `StatePlayerRecyclerViewAssembler` (PR #6126).
- **Hidden for questions:** The progress bar is only shown for explorations, not practice questions. The assembler already distinguishes these via `PlayerFeatureSet` configuration.

Feature 2: Study Guide Fragment

- **New package `app/topic/studyguide/`** (similar to `app/topic/revisioncard/`) with: `StudyGuideActivity`, `StudyGuideFragment`, `StudyGuideFragmentPresenter`, `StudyGuideViewModel`.
- **`StudyGuideFragment`:** Uses a `RecyclerView` with `BindableAdapter` supporting two item types via `MultiTypeBuilder`:
 - **`SECTION_HEADING`:** Bold, 18sp text from `SubtitledUnicode`, announced as header for TalkBack.
 - **`SECTION_CONTENT`:** 16sp HTML-rendered text from `SubtitledHtml`, supporting concept card links, images, and math via `HtmlParser`.
- **Toolbar:** Brown (#BE563C) matching the existing revision card pattern.
- **Navigation cards:** Previous/Next subtopic navigation at the bottom, with content descriptions (e.g., "Previous: What Is a Fraction?").
- **Routing:** `TopicRevisionFragmentPresenter` checks the `enable_study_guides` flag – if ON, launches `StudyGuideActivity`; if OFF, launches `RevisionCardActivity` as before.

Feature 3: Worked Example Tag Handler

- **New `WorkedExampleTagHandler`** implementing `CustomHtmlContentHandler.CustomTagHandler`. Registered in `HtmlParser.computeCustomTagHandlers()` as `handlersMap[CUSTOM_WORKED_EXAMPLE_TAG] = workedExampleTagHandler`.
- **Parsing:** Extracts `question-with-value` and `answer-with-value` JSON attributes (same JSON extraction pattern used by `ImageTagHandler` with `getJsonStringValue()`). Inner HTML is parsed via a new `CustomHtmlParser` interface injected into `handleTag()`.
- **Rendering:** Appends styled spans to the `Editable` output:
 - "Question:" as `StyleSpan(Typeface.BOLD)`
 - Question content as parsed HTML spans
 - "Answer:" as `StyleSpan(Typeface.BOLD)`
 - Answer content as parsed HTML spans
 - Block spacing via `LeadingMarginSpan` and paragraph breaks

- **Accessibility:** `handleTagForContentDescription()` outputs "Question: [text]. Answer: [text]." in linear reading order.
- **No-op when disabled:** When `enable_worked_examples` flag is OFF, a no-op handler is registered that silently drops the tag (same as how concept cards are conditionally registered).
- **"Question:" and "Answer:" labels:** Defined as translatable string resources in `strings.xml`, extracted via `resourceHandler` so they appear in the user's selected language.

[Android only] Test data changes

Lesson progress: PR [#6185](#) already added `is_checkpoint: true` to 2 states in `test_exp_id_2.textproto` and `test_exp_id_2.json`. I will add `is_checkpoint: true` to at least one more state (for a minimum of 3 checkpoints) to make the progress bar more demonstrative in tests. Create one exploration with no checkpoints to test the hidden-progress-bar case.

Study guide: Create test study guide assets (`test_topic_id_0_1.textproto` and `.json`) with sections format. Keep existing revision card test assets unchanged for backward compatibility testing.

Worked examples: Add `<oppia-noninteractive-workedexample>` tags to test concept card HTML content. Include edge cases: empty question/answer, nested HTML (bold, images), malformed attributes.

[Android only] Testing library changes

New test helpers: `StudyGuideTestHelper` for creating test `StudyGuideRecord` protos, and `CheckpointProgressSubject` Truth subject for asserting on progress values. These follow the existing pattern of `ExplorationCheckpointTestHelper` and `AsyncResultSubject`.

Feature flag test utilities: New `forceEnable*()` methods added to `TestPlatformParameterModule`, one per new flag, following the existing pattern (`forceEnableLearnerStudyAnalytics`, `forceEnableSpotlightUi` etc). No new test modules are introduced.

Test coverage mapping

Below is the mapping from each class I touch (new or modified) to the test file that covers it. Classes that can be tested in isolation get their own test file. Classes that are tightly coupled to their consumer are covered through the consumer's test instead. This follows the existing pattern in the codebase, for example `HintHandlerProdImpl` has its own `HintHandlerProdImplTest` while `StatePlayerRecyclerViewAssembler` is covered through `StateFragmentTest`.

Feature 1: Lesson Progress

Class / change	New / Modified	Test file	New / Existing
<code>StateGraph.computeCheckpointCount()</code> , Dijkstra pathfinding and edge cases	Modified	<code>StateGraphTest</code>	New
<code>StateDeck.computeCompletedCheckpointCount()</code> helper	Modified	<code>StateDeckTest</code>	New

StateGraph and StateDeck behavioral wiring (forward, backward, flashback, no-checkpoint exploration)	Modified	ExplorationProgressControllerTest	Existing
ExplorationProgressController.getCurrentEphemeralState() passes total checkpoint count	Modified	ExplorationProgressControllerTest	Existing
StatePlayerRecyclerViewAssembler.addLessonProgressItem() and PlayerFeatureSet.lessonProgressSupport	Modified	StateFragmentTest	Existing
StateFragmentPresenter rendering of new ViewType	Modified	StateFragmentTest	Existing
LessonProgressViewModel (extends StateItemViewModel)	New	StateFragmentTest	Existing
CheckpointProgress proto and EphemeralState.checkpoint_progress field	New	ExplorationProgressControllerTest (asserted via CheckpointProgressSubject)	Existing
enable_lesson_progress_visualization feature flag	New	Exercised by all tests above that toggle the flag via TestPlatformParameterModule	Existing

Note: StateItemViewModel subclasses (ContinueInteractionViewModel, ContentViewModel etc) do not have their own test files in this codebase. They are covered through StateFragmentTest. LessonProgressViewModel follows the same pattern.

Note: Layout XMLs (lesson_progress_indicator_item.xml, modified continue_interaction_item.xml etc) are exercised through Espresso assertions in StateFragmentTest, no separate test files.

Feature 2: Study Guide

Class / change	New / Modified	Test file	New / Existing
StudyGuideRetriever	New	TopicControllerTest	Existing
TopicController.getStudyGuide()	New method	TopicControllerTest	Existing
StudyGuideFragment, StudyGuideFragmentPresenter,	New	StudyGuideFragmentTest	New

StudyGuideViewModel			
StudyGuideActivity (lifecycle, intent extras, configuration changes)	New	StudyGuideActivityTest	New
StudyGuideActivity (Robolectric-only local test)	New	StudyGuideActivityLocalTest	New
TopicRevisionFragmentPresenter flag-based routing	Modified	TopicRevisionFragmentTest	Existing
RevisionCardFragment / RevisionCardActivity (untouched, regression check with flag OFF)	Untouched	RevisionCardFragmentTest and RevisionCardActivityLocalTest	Existing
StudyGuide / StudyGuideRecord / StudyGuideSection / EphemeralStudyGuide proto messages	New	TopicControllerTest (using StudyGuideTestHelper fixtures)	Existing
enable_study_guides feature flag	New	Exercised by all tests above that toggle the flag	Existing

Note: StudyGuideRetriever follows the RevisionCardRetriever pattern. RevisionCardRetriever also does not have a dedicated test file and is covered through TopicControllerTest.

Feature 3: Worked Examples

Class / change	New / Modified	Test file	New / Existing
WorkedExampleTagHandler – parses question/answer attributes, builds Spannable	New	WorkedExampleTagHandlerTest (mirrors ConceptCardTagHandlerTest and ImageTagHandlerTest)	New
CustomHtmlParser interface (default impl in CustomHtmlContentHandler.fromHtml())	New	WorkedExampleTagHandlerTest (consumer view) and CustomHtmlContentHandlerTest (impl view)	New + Existing
CustomHtmlContentHandler.handleTag() and handleTagForContentDescription() signature change	Modified	CustomHtmlContentHandlerTest	Existing
HtmlParser.computeCustomTagHandlers()	Modified	HtmlParserTest	Existing

registers new handler			
<code>enable_worked_examples</code> feature flag	New	Flag-OFF behavior asserted directly in <code>WorkedExampleTagHandlerTest</code>	Existing
New translatable strings (<code>Question:</code> , <code>Answer:</code> in <code>strings.xml</code>)	New	Exercised by <code>WorkedExampleTagHandlerTest</code>	Existing

[Android only] Script & CI changes

- Add new test targets to Bazel BUILD files for `StudyGuideFragmentTest`, `WorkedExampleTagHandlerTest`, `LessonProgressViewModelTest`.
- Add any new test files that use `java.util.Locale` to the exemption list in `file_content_validation_checks.textproto` (same as I did in PR [#6102](#)).
- No new CI workflows needed — existing test infrastructure covers all new test targets.

Documentation changes

- Add inline KDoc to new public classes and methods following the existing codebase style.

Metrics Plan

Event (see PRD)	Event parameters (see PRD)	Do we already record the event + parameters? • If so, please link to the corresponding code on GitHub. • If not, describe the changes needed to do so.
<code>finish_exploration_context</code> ("complete_exploration")	<code>ExplorationContext</code> : topic_id, story_id, exploration_id, session_id, exploration_version, state_name, learner_id	Yes, already recorded. Logged by <code>LearnerAnalyticsLogger.ExplorationAnalyticsLogger.logFinishExploration()</code> in LearnerAnalyticsLogger.kt . Called from ExplorationProgressController.kt (line ~1474). This event measures whether lesson progress visualization improves exploration completion rates.
<code>exit_exploration_context</code> ("leave_exploration")	<code>ExplorationContext</code> : topic_id, story_id, exploration_id, session_id, exploration_version,	Yes, already recorded. Logged by <code>LearnerAnalyticsLogger.ExplorationAnalyticsLogger.logExitExploration()</code> in LearnerAnalyticsLogger.kt . A decrease in exit-before-completion events would

	state_name, learner_id	indicate the progress bar helps learners persist.
open_revision_card	RevisionCardContext: topic_id, subtopic_index	Yes, already recorded. Logged by OppiaLogger .createOpenRevisionCardContext() in OppiaLogger.kt, called from RevisionCardFragmentPresenter.kt (line ~123). The same event will be logged from StudyGuideFragmentPresenter to measure study guide engagement using the existing RevisionCardContext.

No new analytics events are introduced by this project. All three features operate on locally-loaded content and their impact is measurable through the existing event pipeline above. Since feature flag states are logged per-session via FeatureFlagsLogger, these existing events can be compared with the corresponding feature flags on vs. off to measure whether the features improve completion rates or study guide engagement.

Implementation Plan

Milestone Table (include both PRs and other actions that need to be taken prior to launch)

Milestone 1

Key objective for this milestone: *{Lesson progress indicator and study guide both fully implemented end-to-end (proto -> domain -> UI) and verified with tests. Both features are gated behind feature flags and QA tested.}*

No.	Description of PR / action	Prerequisite steps	Target date	
			PR creation	PR merge
1	PR: Lesson progress full domain layer. Add <i>CheckpointProgress</i> proto message and <i>checkpoint_progress</i> field on <i>EphemeralState</i> in <i>exploration.proto</i> . Add <i>enable_lesson_progress_visualization</i> feature flag in <i>FeatureFlagConstants.kt</i> (+ Dagger qualifier, <i>FeatureFlagsLogger</i> registration). <i>StateGraph.computeTotalCheckpointCount()</i> with pathfinding via <i>labelled_as_correct</i> . <i>StateDeck</i> checkpoint tracking, populate <i>CheckpointProgress</i> in <i>getCurrentEphemeralState()</i> . <i>PlayerFeatureSet.lessonProgressSupport</i> . Test exploration data with <i>is_checkpoint</i> fields + no-checkpoint exploration. New <i>StateGraphTest</i> covering Dijkstra edge cases (single path, multiple terminal states, zero checkpoints, branching). New <i>StateDeckTest</i> covering	None	May 26	Jun 9

	computeCompletedCheckpointCount across navigation modes. Extended ExplorationProgressControllerTest covering end-to-end checkpoint progress wiring (forward, backward, flashback, no-checkpoint exploration).			
2	PR: Study guide full domain layer. Add StudyGuideSection, StudyGuideRecord, StudyGuide, EphemeralStudyGuide proto messages to topic.proto. Add enable_study_guides feature flag (+ FeatureFlagsLogger). Create StudyGuideRetriever following RevisionCardRetriever pattern. Add getStudyGuide() to TopicController. Test study guide assets (.textproto and .json). Unit tests.	None	May 26	Jun 9
3	PR: Lesson progress full UI + tests. LessonProgressViewModel (completedCount, totalCount, progressText – progress bar width determined by layout). lesson_progress_indicator_item.xml, custom drawable for checkpoint nodes (completed/in-progress/incomplete). ViewType.LESSON_PROGRESS_INDICATOR, register binder in StatePlayerRecyclerViewAssembler, translatable “X of N” string, TalkBack content description. StateFragmentTest cases: progress bar visible with checkpoints, correct count after advancing, hidden when no checkpoints, hidden for questions, hidden when flag OFF. Espresso matchers. LessonProgressViewModel follows the StateItemViewModel convention of being tested through StateFragmentTest, no separate test file..	PR 1	Jun 4	Jun 19
4	PR: Study guide full UI + routing + tests. Create app/topic/studyguide/ package with StudyGuideActivity, StudyGuideFragment, StudyGuideFragmentPresenter, StudyGuideViewModel. RecyclerView with BindableAdapter.MultiTypeBuilder for section heading + section content items. Brown toolbar (#BE563C). Conditional routing in TopicRevisionFragmentPresenter: if flag ON → launch StudyGuideActivity, else → launch RevisionCardActivity. Previous/Next subtopic navigation. Backward compatibility. StudyGuideFragmentTest: section headings rendered correctly, section content HTML parsed with images/math/concept cards, translations apply via TranslationController, previous/next navigation works, flag-off falls back to revision card. Espresso + Robolectric.	PR 2	Jun 4	Jun 19

–	Lesson progress indicator and study guide features both fully implemented. User journeys 1–11 from the testing plan can be fully tested by turning on the <i>enable_lesson_progress_visualization</i> and <i>enable_study_guides</i> feature flags.			
--	Action: M1 QA testing + bug fixes. Manual testing of both features on device. Respond to code review feedback on all open PRs. CI buffer.	PR 1–4	--	Jun 26
--	Demo to PMs: Lesson progress showing "X of N" with back navigation, checkpointing, and flashback handling. Progress hidden for non-checkpoint explorations and for questions. Study guide showing section headings and content when flag ON, falling back to revision card when flag OFF.	PR 1–4	Jun 26	

Note: PRs 1 & 2 are independent and worked in parallel. Coding for PRs 3 & 4 begins while PRs 1 & 2 are in review.

Milestone 2

Key objective for this milestone: *{Worked example tag handler complete and launched. Bug fixes for all three features from M1 feedback. All three feature flags turned ON for production.}*

No.	Description of PR / action	Prerequisite steps	Target date	
			PR creation	PR merge
5	PR: <i>WorkedExampleTagHandler</i> full implementation + tests + flag. Handler implementing <i>CustomHtmlContentHandler.CustomTagHandler</i> . Parse <i>question-with-value</i> and <i>answer-with-value</i> JSON attributes. <i>CustomHtmlParser</i> interface that re-invokes the full tag handler pipeline minus the worked example handler to support nested custom tags (images, math, concept cards). Render bold "Question:"/"Answer:" labels as styled spans (<i>StyleSpan(BOLD)</i> , <i>LeadingMarginSpan</i>). Register globally in <i>HtmlParser.computeCustomTagHandlers()</i> . Add <i>enable_worked_examples</i> feature flag (+ <i>FeatureFlagsLogger</i>). Translatable strings. Accessibility handler. No-op handler when flag OFF. <i>WorkedExampleTagHandlerTest</i> : renders labels, handles nested HTML (bold, images, math, concept cards), accessibility content description	None	Jul 18	Aug 1

	correct, empty/malformed attributes handled. <i>HtmlParserTest</i> integration.			
–	<i>Worked example feature fully implemented. User journeys 12–16 from the testing plan can be fully tested by turning on the <i>enable_worked_examples</i> feature flag.</i>			
–	Action: QA testing for all three features. Manual testing on multiple devices. End-to-end testing of full learner flow. Report bugs.	PR 5	Aug 1	Aug 15
6	PR: Bug fixes for all three features based on M1 QA feedback and PM review. Address visual polish, edge cases, and design feedback.	PR 3, 4, 5	Aug 8	Aug 25
7	PR: Enable all three feature flags. Set defaults to true for <i>enable_lesson_progress_visualization</i> , <i>enable_study_guides</i> , <i>enable_worked_examples</i> . Full integration verification with all three features running together.	PR 6	Aug 25	Sep 1
--	Demo to PMs: Worked examples rendering "Question:"/"Answer:" blocks in concept cards. All three features enabled in a full learner flow: exploration with progress bar → topic page → study guide → concept card with worked examples. Lesson progress and study guide fully production ready with M1 feedback addressed.	PR 5–7	Sep 2	

Feature Flags

#	Name of feature flag	What happens when the flag is turned on	Needs sync between Android / Web?	Key stakeholders
1.	<i>android_enable_lesson_progress_visualization</i> Qualifier: <i>@EnableLessonProgressVisualization</i> Default: <i>false</i>	A horizontal checkpoint progress bar appears below Continue/Next buttons during explorations. Shows "X of N" with filled/empty circle nodes. The bar is rendered by <i>StatePlayerRecyclerView-Assembler</i> via a new <i>LessonProgressViewModel</i> . When OFF, no progress	Resolved. The <i>is_checkpoint</i> field is now available in both proto and JSON formats (PR #6185 , closing #6103). No external dependencies remain enabling this flag is purely an Android-side	@MohitGupta121 (mentor), @BenHenning (Android TL / interim product area lead), release coordinator @adhiambooyier, PM team (approve)

		indicator is shown and the exploration UI is unchanged. The <code>CheckpointProgress</code> data is still computed in <code>StateDeck</code> but not rendered, so turning the flag ON later requires no domain layer changes.	decision.	progress bar UX before launch)
2.	<code>android_enable_study_guides</code> Qualifier: <code>@EnableStudyGuides</code> Default: <code>false</code>	Clicking a subtopic in the Topic Revision tab opens <code>StudyGuideActivity</code> instead of <code>RevisionCardActivity</code> . Study guides show a sectioned layout (heading + content pairs in a <code>RecyclerView</code>) instead of a single flat HTML block. The routing switch happens in <code>TopicRevisionFragmentPresenter</code> . When OFF, <code>RevisionCardActivity</code> launches as before — zero behavioral change.	Yes, the app needs study guide assets exported in the <code>ACTIVITY_TYPE_SUBTOPIC_WITH_STUDY_GUIDE</code> format. This endpoint already exists on web (created by the GSoC 2025 web team for #5844). Android must switch its asset download script to request this format instead of <code>ACTIVITY_TYPE_SUBTOPIC</code> . The flag can be turned on once the bundled assets use the new format.	@MohitGupta121 (mentor), @BenHenning (Android TL / interim product area lead), release coordinator @adhiambooyier, GSoC 2025 web team (confirm export format stability), PM team (review study guide UX)
3.	<code>android_enable_worked_examples</code> Qualifier: <code>@EnableWorkedExamples</code> Default: <code>false</code>	<code>HtmlParser</code> registers a <code>WorkedExampleTagHandler</code> for <code><oppia-noninteractive-workedexample></code> tags. Concept card HTML containing this tag renders bold "Question:"/"Answer:" labels with styled content blocks. When OFF, a no-op handler is registered that silently ignores the tag (same pattern as the conditional <code>supportsConceptCards</code> check in <code>HtmlParser.computeCustomTagHandlers()</code>).	No sync needed. The <code><oppia-noninteractive-workedexample></code> tag is already used in web content (#5963). The tag exists in the HTML that is already exported — Android just needs a handler to render it. Turning this flag on is purely an Android-side decision.	@MohitGupta121 (mentor), @BenHenning (Android TL / interim product area lead), release coordinator @adhiambooyier, PM team (approve worked example rendering)

All three flags default to **false** and are flipped to true together in PR 7 of Milestone 2 after QA passes. Behavior when each flag is OFF:

- **Lesson progress:** **CheckpointProgress** is still computed in **StateDeck** for test stability, but the assembler does not render the progress bar. Existing exploration player is unchanged.
- **Study guides:** **TopicRevisionFragmentPresenter** keeps launching **RevisionCardActivity** as before. The new **StudyGuideActivity** is never reached.
- **Worked examples:** the handler is not registered in **HtmlParser**, so the tag passes through as an unknown element and is silently dropped. Matches current production behavior.

The three flags are independent. The one non-trivial interaction is **enable_worked_examples = ON** with **enable_study_guides = OFF**. In that case worked-example tags still render inline inside the legacy revision-card content, because the worked-example handler is registered in the shared **HtmlParser** used by both the old **RevisionCardFragment** and the new **StudyGuideFragment**.

Future Work

Note: *This section is mainly for reference (since items listed here won't be part of the GSoC project). Proposals will primarily be evaluated based on the implementation plan above.*

There are a few things that came up while working on this proposal that I think would be worth doing after GSoC, but they are out of scope for this project:

- **Voiceover playback for study guides.** The Android app has working voiceover infrastructure, **AudioUiManager** and **AudioFragmentPresenter** handle playback in explorations, and the **SubtopicRecord** proto already has **recorded_voiceover** mappings. However, voiceovers for non-exploration content like study guides do not exist yet on the web side. The web team is currently working on introducing automated voiceovers, which would eventually produce audio for study guide sections. Once that content is available, the Android side would need to wire up per-section playback by adding an audio control to each section item in the **RecyclerView**, which is more involved than the single audio bar used in explorations since each section has independent content.
- **Collapsible worked examples.** This project renders worked examples as inline Question/Answer text because the web's collapsible "Example"/"Solution (click to reveal)" UI cannot be replicated using Android's current HTML-to-Spannable pipeline. The project description notes this is a deceptively challenging problem, implementing interactive collapsible sections within a **TextView** would require either a **ClickableSpan**-based toggle or inflating custom views into the content flow, both of which risk badly breaking the accessibility experience (TalkBack focus order, touch target sizing, state announcements). Solving this properly likely requires reworking how **CustomHtmlContentHandler** processes tags, which is outside the scope of this project and would need careful design input from the accessibility and product teams.
- **Topic-level progress overview.** **StoryProgressController** already tracks which explorations within a story are completed, and **StorySummaryViewModel** displays a **storyPercentage** for each individual story. But there is no aggregated progress indicator at the topic level showing overall completion across all stories. This could reuse the checkpoint progress drawable from this project with

different data, completed explorations out of total explorations in the topic and would give learners a high-level view of where they stand before diving into individual stories.